UNIVERSITY OF EASTERN PANGASINAN

DESPERA MERCHANDISE

FINAL SYSTEM DOCUMENTATION

B2C E-COMMERCE SYSTEM

DESPERA MERCHANDISE

A web-based business-to-customer merchandise storefront

Group Members	Noces, Kenneth Adrian Oria
Section	BSIT-2F
Subject	Information Management 1
Instructor	Samuel S. Mindaro, Jr.
Date of Submission	November 2026

Documentation Map

Documentation Map 1
Background of the System 2
Problem Statement 2
Objectives 2
Scope and Limitations 3
Significance of the System 3
System Description 4
Target Users 4
Functional Requirements 4
Non-Functional Requirements 5
Use Case Diagram 6
System Flowchart 7
Entity-Relationship Diagram (ERD) 8
Database Design 8
Important Data Rules 9
Wireframes 9
Technology and Runtime 10
What the Group Developed 14
What Was Achieved 14
Problems Encountered and Solutions Implemented 14
Lessons Learned 14
Possible Future Improvements 15

SECTION I

INTRODUCTION

Background of the System

Despera Merchandise is a fan-made academic demonstration storefront for a curated collection of anime-inspired apparel, accessories, and collectibles. The project implements a business-to-customer (B2C) e-commerce workflow: customers explore a product catalog, manage a personal cart, provide a delivery location, place an order, and review order history. Administrators maintain products and categories, review orders, view store summaries, manage users, and prepare printable reports.

The application combines a responsive browser interface with a PHP API and a relational MySQL database. The user interface uses a restrained warm palette and Bootstrap-based layout. Its design notes emphasize a readable mobile/desktop experience, gentle contrast, clear iconography, and compact navigation.

Problem Statement

A small merchandise shop needs a consistent way to publish products, show stock, accept customer selections, retain order details, and support basic administration. Keeping these records in separate manual lists makes it harder to keep product availability, cart contents, order totals, payment status, and customer history aligned. This project addresses the need for one connected demonstration system that supports these routine customer and administrative tasks.

Objectives

General objective: develop a web-based B2C merchandise system that demonstrates the essential customer shopping lifecycle and administrator catalog and order management.

- Provide account registration, sign-in, session persistence, profile and delivery-location management.
- Allow customers to browse, search, inspect, and add available merchandise to a persistent cart.
- Create orders from cart contents, retain item and price snapshots, and expose order history and status.
- Give administrators catalog/category CRUD, order and user management, inventory indicators, dashboards, and printable reports.
- Use database relationships and server-side authorization to keep customer and administrative operations tied to authenticated accounts.

- Demonstrate an optional AI-assisted customer-service experience with stored conversation history and a shared request cooldown.

Scope and Limitations

Scope: the application covers customer and admin accounts; product/category catalog; product search and stock visibility; user carts; checkout and delivery details; pending, paid, delivered, and cancelled order states; sales and inventory summaries; printable reports; and optional AI-based customer service.

Limitations: checkout/payment is a local demonstration. The customer manually confirms a GCash payment; no payment provider verifies, authorizes, settles, refunds, or reconciles real funds. The AI assistant depends on optional Gemini access, has a shared one-request-per-minute cooldown, and may generate incorrect answers. The repository is configured for academic/local demonstration and static frontend preview, not production payment processing or a production security certification. A first administrator is provisioned manually through the database. Live test evidence and institutional metadata must be supplied by the group.

Significance of the System

For customers, the system demonstrates a clear self-service path from catalog discovery to order follow-up. For administrators, shared product, category, stock, customer, and order records reduce repetitive tracking and provide a concise view of store activity. For the student developer, the project connects front-end interaction, REST-style routing, session-based authorization, relational design, and deployment configuration in one applied CRUD system.

SECTION II

SYSTEM ANALYSIS AND DESIGN

System Description

The client is a set of HTML pages, CSS stylesheets, and JavaScript modules served as static files. Modules call PHP API routes with credentialed requests. The PHP application uses PDO to access MySQL and separates route handling from repositories/services. Podman Compose provides the local PHP/Apache server, MySQL database, and optional Adminer/phpMyAdmin management tools. The browser selects a local or hosted API origin using client/config.js. When configured, the API can call Gemini for the customer-service assistant; the API key remains server-side. The hosted frontend is available at https://despera-merchandise.42web.io/client/; the public API is served on the same host under /api/.

Layer	Project components and responsibility
Presentation	client/ HTML, CSS, Bootstrap, JavaScript modules; customer and admin interfaces.
API	server/index.php and server/api/; JSON routes, input handling, authentication, and authorization.
Services / data	server/services/ repositories, validation, sessions, cart/order operations, and PDO access.
Persistence	MySQL schema in server/schema/db_create.sql; foreign keys, checks, indexes, and migrations.
Local operations	infra/ Podman Compose; scripts/ setup, start, and stop helpers; database volume.
Optional external service	Gemini API for assistant and login greeting; requires a server-side key and is not required for core shopping.

Target Users

User	Access and main tasks
Customer (role_id 2)	Register/sign in; browse, search, and view merchandise; manage cart/profile/location; create and review orders; cancel an eligible pending order; use customer-service chat.
Administrator (role_id 1)	Review sales/inventory dashboard and reports; create, view, update, and delete product/category records; review and transition orders; manage users; use role-aware assistant context.

Role checks are enforced at protected API routes. Public registration creates customer accounts; initial admin provisioning is a documented setup operation.

Functional Requirements

ID	Requirement
FR-01 Accounts	Register a customer; authenticate by username/password; establish and revoke a server-side session; optionally remember a session.

ID	Requirement
FR-02 Product discovery	Read product/category information, search/filter products, inspect details and stock visibility.
FR-03 Cart and profile	Maintain a user-owned cart, change item quantity, remove items, and save delivery location/profile details.
FR-04 Orders	Create an order from a non-empty cart; preserve item details and price; calculate total; list customer orders; cancel only eligible pending orders.
FR-05 Payment/status	Demonstrate manual GCash confirmation; prevent repeat/non-pending payment; check stock and decrement atomically; allow valid admin order-state transitions.
FR-06 Administration	Admin CRUD for products, categories, and users; review orders and statuses; search/filter management views.
FR-07 Dashboard/reports	Summarize sales, paid/pending orders, products, customer counts, low stock, recent orders, and printable reports.
FR-08 Customer service	Persist per-account conversations; show role-appropriate store context; enforce shared AI request cooldown.

Non-Functional Requirements

Quality	Design or implementation evidence
Usability	Warm, restrained visual language; responsive Bootstrap components; labeled forms, status messaging, and role-aware navigation.
Security	Passwords stored as hashes; random session tokens stored as SHA-256 hashes; HttpOnly session cookie; server-side role/ownership checks; PDO prepared queries in repositories.
Data integrity	Foreign keys and cascades; unique keys; positive price/quantity and nonnegative stock checks; transaction/row locking for payment and stock decrement.
Compatibility	HTML/CSS/JavaScript client and PHP 8.4/Apache API with MySQL 8; local Compose configuration and static-host frontend preview.
Maintainability	Routes, repositories/services, schema, migrations, client pages, API modules, styles, and deployment scripts are separated by responsibility.
Availability	Core catalog, cart, checkout demonstration, and administration are independent of optional Gemini access; hosting/API/network availability remains a dependency.

Use Case Diagram

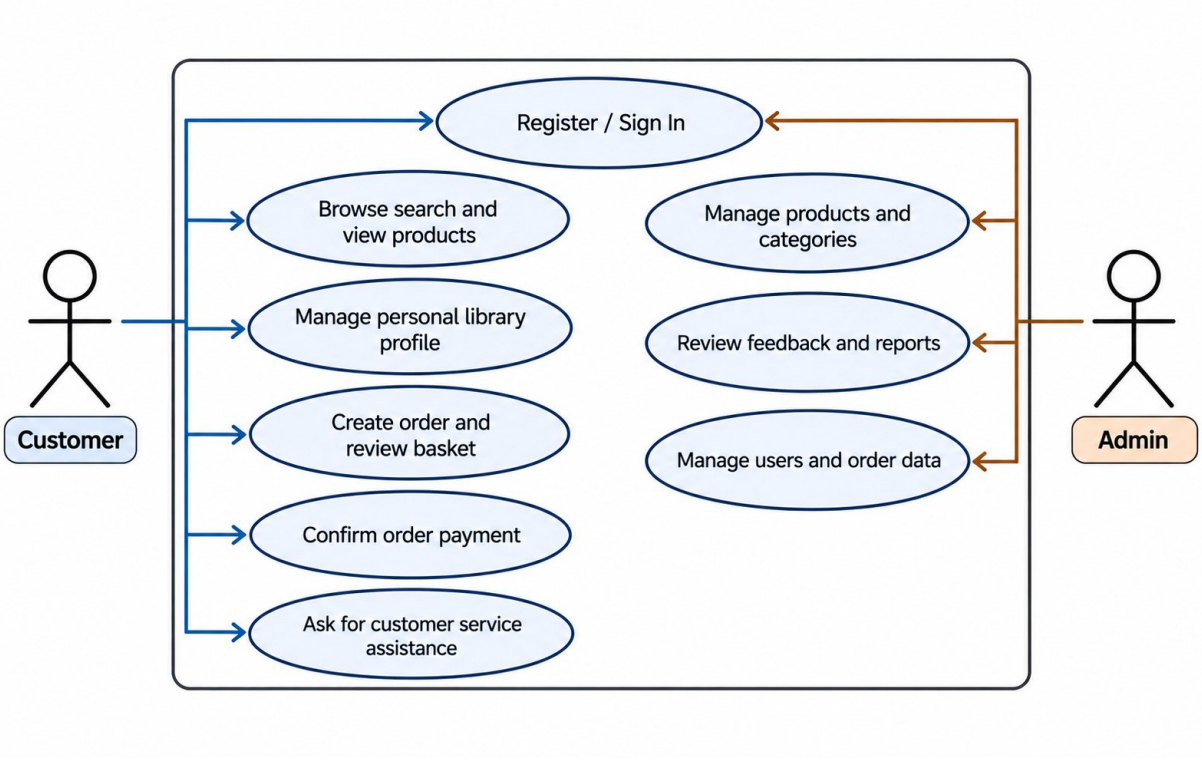


Figure 1. Customer and administrator actions supported by the system.

System Flowchart

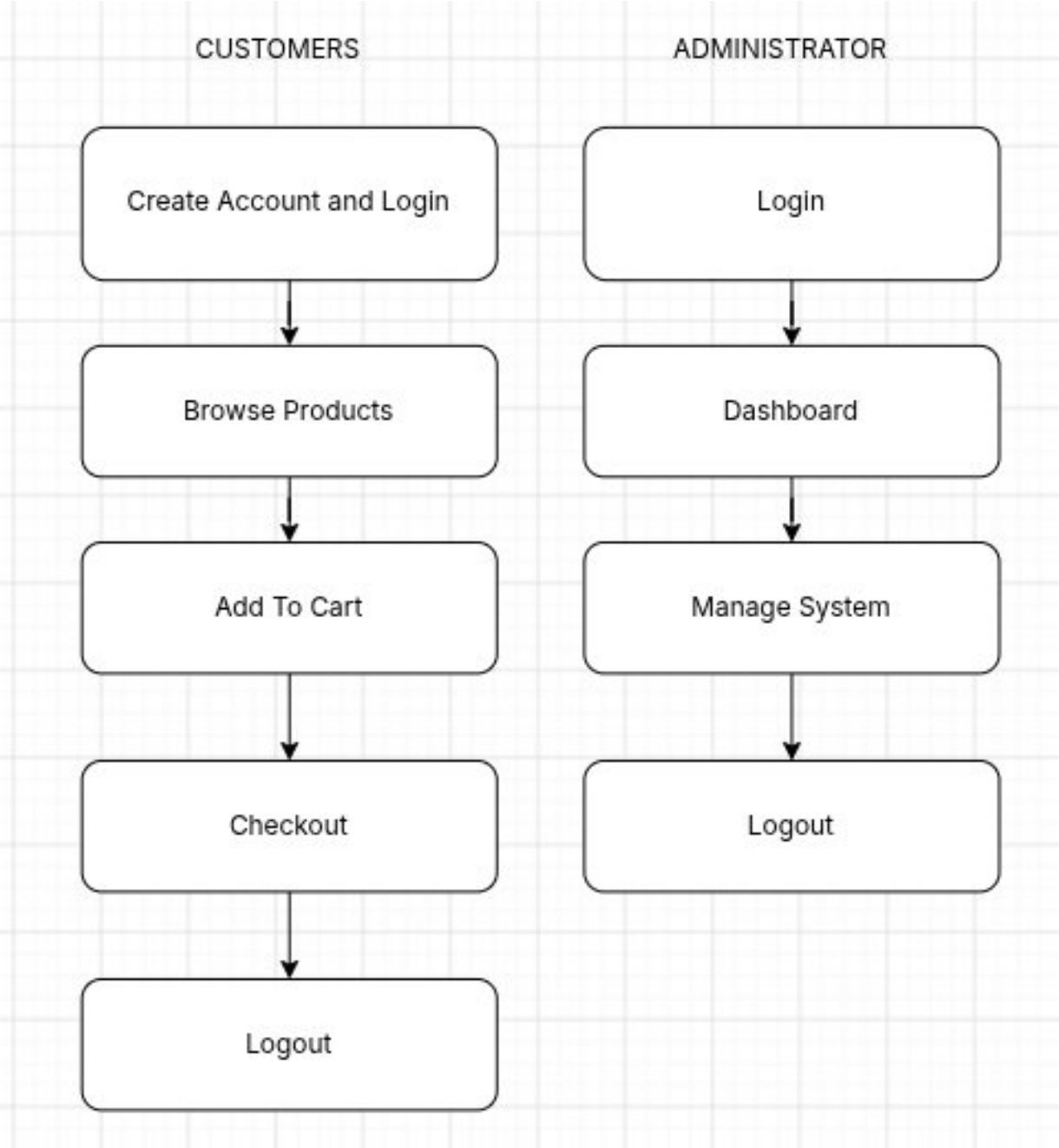


Figure 2. Customer checkout and administrator workflows. Payment confirmation is a demonstration only.

SECTION II

SYSTEM ANALYSIS AND DESIGN — CONTINUED

Entity-Relationship Diagram (ERD)

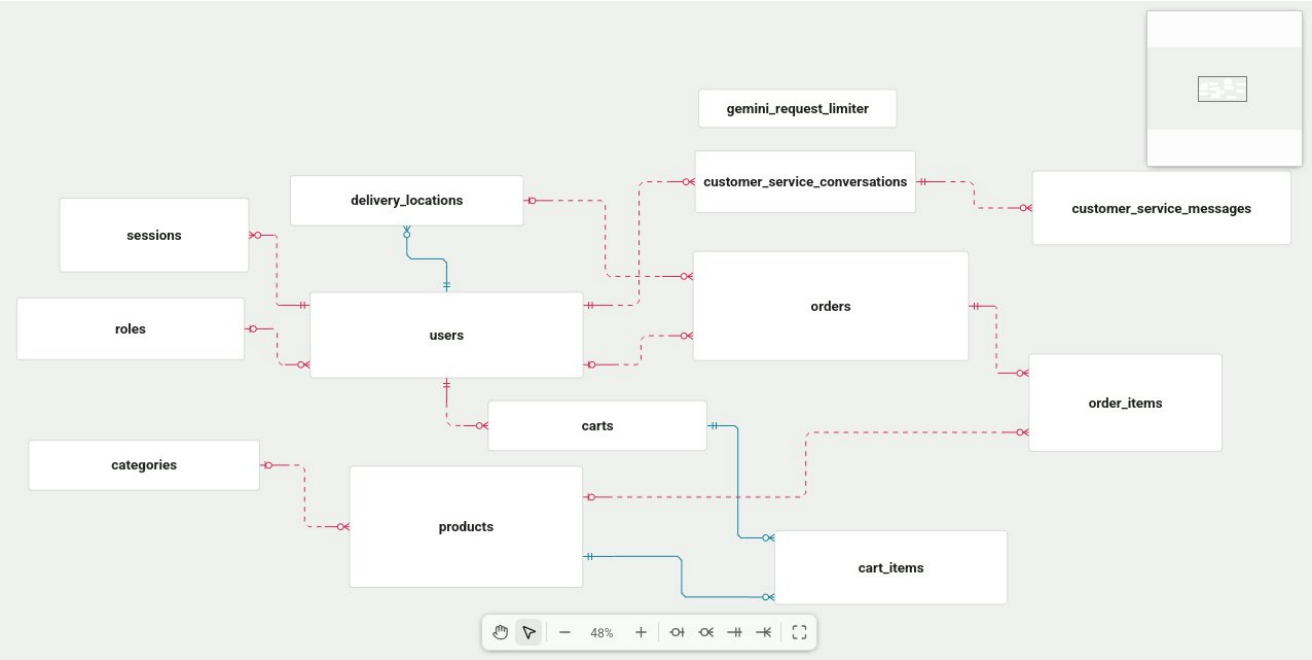


Figure 3. Project database entities and key relationships. CS denotes customer service.

Database Design

The schema uses primary keys for entity identity and foreign keys for account, catalog, cart, order, delivery, and customer-service relationships. Usernames and category/product names are unique. A cart belongs to one user; each cart item is unique per cart/product pair. Order items retain product name, description, and unit price as purchase-time snapshots so later catalog changes do not rewrite the order record.

Table	Primary key	Purpose and key relationships
roles	id	Role names/descriptions; referenced by users.
users	id	role_id → roles; unique username; password_hash and timestamps.
sessions	id	user_id → users; unique token_hash; remember flag and expiry.
categories	id	Unique merchandise category name.
products	id	category_id → categories; name, description, positive unit_price, nonnegative stocks, image path.
carts	id	Unique user_id → users; one cart per user.
cart_items	cart_id + product_id	Quantity and update time; references carts and products.
delivery_locations	user_id	One saved location per user; referenced by orders.
orders	id	user_id, location_id, status, payment method, total, created/paid timestamps.

Table	Primary key	Purpose and key relationships
order_items	id	order_id and optional product_id; quantity and snapshot name/description/unit price.
customer_service_conversations	id	Unique user_id; one conversation per user.
customer_service_messages	id	conversation_id; sender, message, timestamp; indexed by conversation/time.
gemini_request_limiter	id	Single shared cooldown row and last_request_at timestamp.

Important Data Rules

- Deleting a user cascades to sessions, carts, delivery location, and customer-service conversation; order ownership is set null to preserve order history.
- Deleting a category sets a product category reference to null; deleting a product removes matching cart rows while order-item product references can be set null.
- Orders cannot be created from an empty cart. Checkout copies cart rows into order items and clears the cart.
- Payment only proceeds from pending status. The server locks the order, validates that each item has enough stock, and decrements inventory within a transaction; failure prevents a successful payment state.
- The order state values used by route logic are pending, paid, delivered, and cancelled. README-listed payment is a manual demo confirmation, not a financial integration.

Wireframes

The following structural wireframe summarizes the administrator product list. The implemented page uses a responsive card grid, product search, add-product action, stock status, edit action, and a delete confirmation.

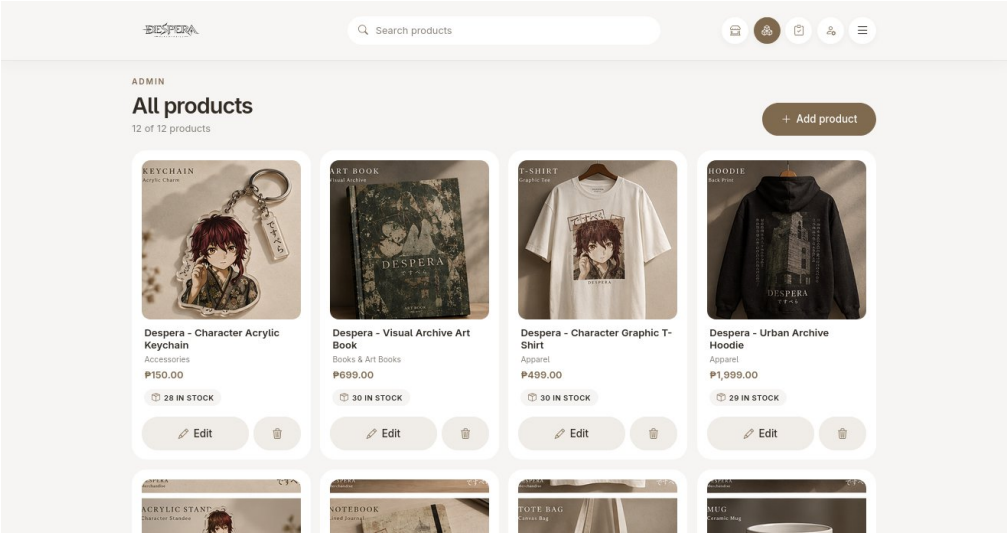


Figure 4. Product administration layout distilled from the implemented page structure.

SECTION III

SYSTEM IMPLEMENTATION

Technology and Runtime

Area	Implementation
Frontend	HTML5, CSS, Bootstrap 5, Bootstrap Icons, and native JavaScript modules.
Backend	PHP 8.4 on Apache; JSON API handlers and repository/service classes.
Database	MySQL 8 with schema and incremental migration scripts.
Local deployment	Podman Compose; server and MySQL containers with Adminer/phpMyAdmin tooling.
Optional AI	Gemini assistant and login greeting; server-side key, conversation persistence, shared cooldown, and fallback notices.

LOGIN

Enter Username and Password with Show password, remember me feature.

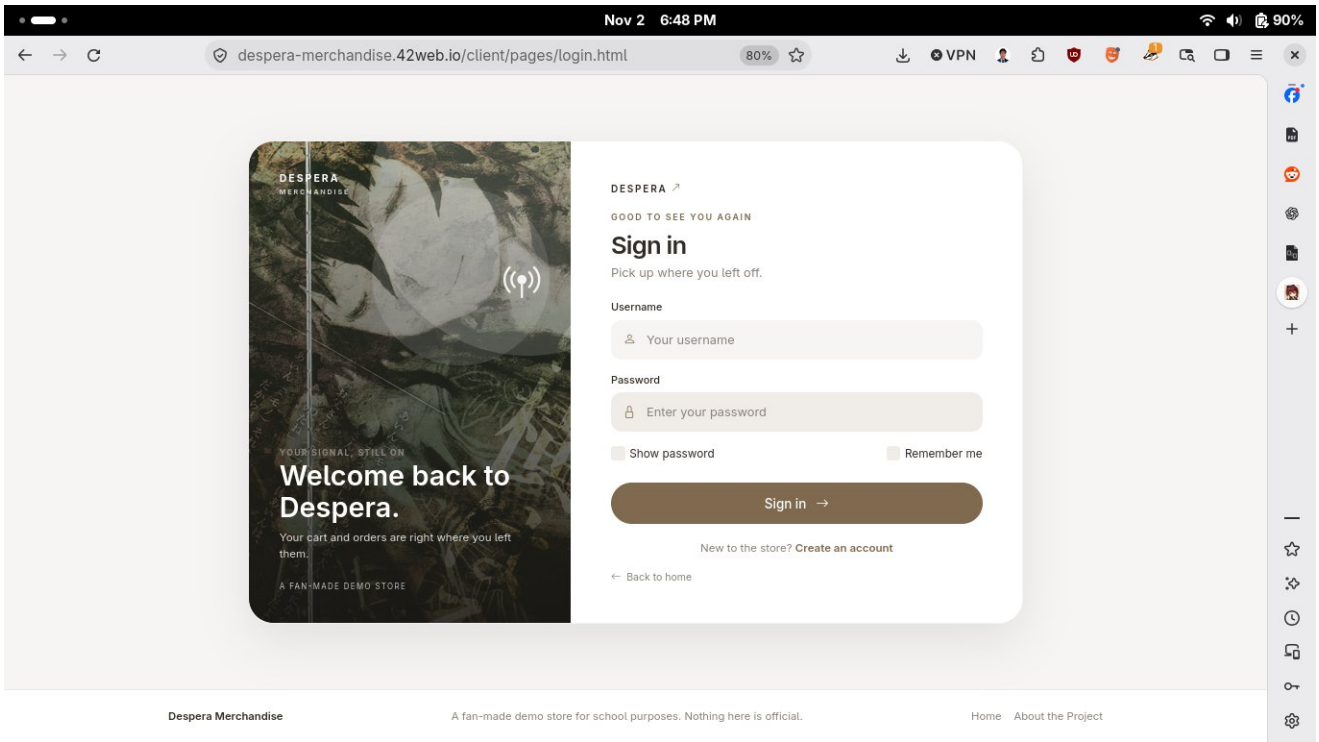


Figure 5. Login Page.

DASHBOARD

View Store current Status with the ability to refresh easily and print reports.

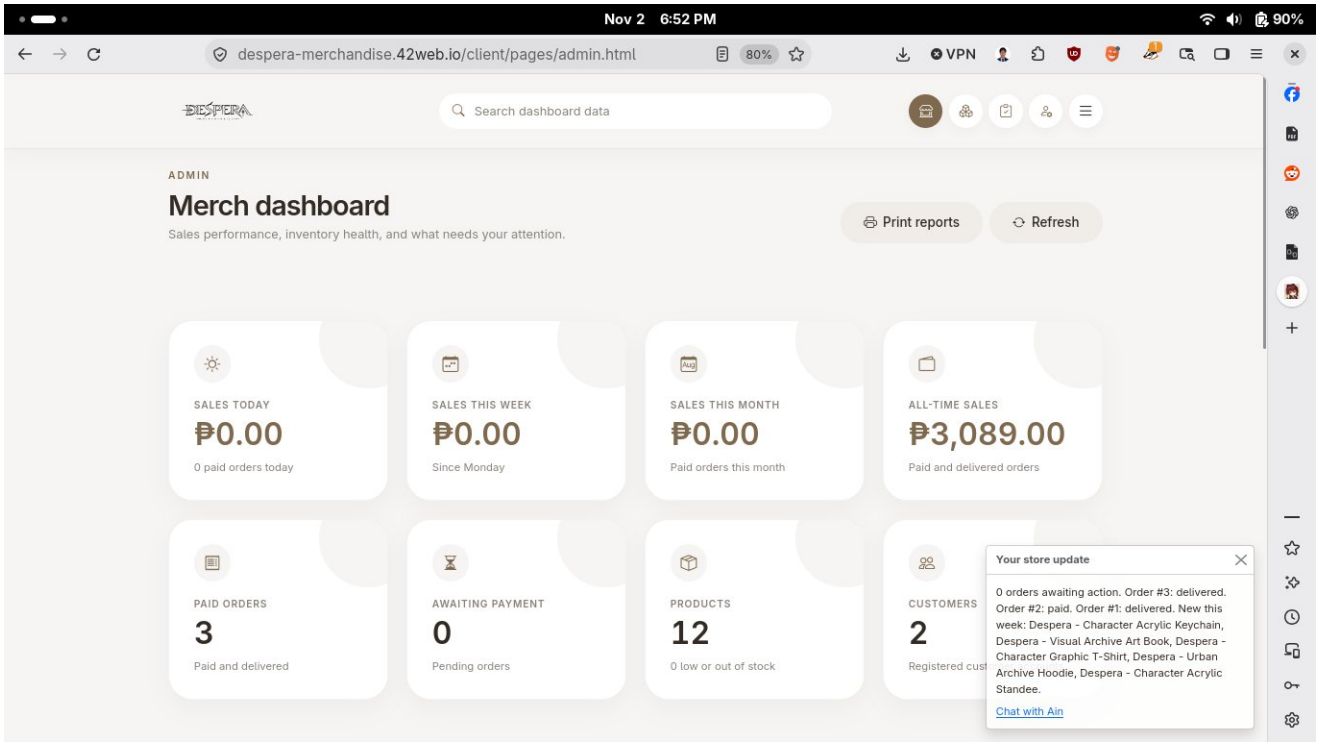


Figure 6. Dashboard page.

VIEW/ADD/DELETE/UPDATE PRODUCT

Navigate to Products page and view products, Click add product to add product, Click Delete Icon to delete product, Click edit button to update product.

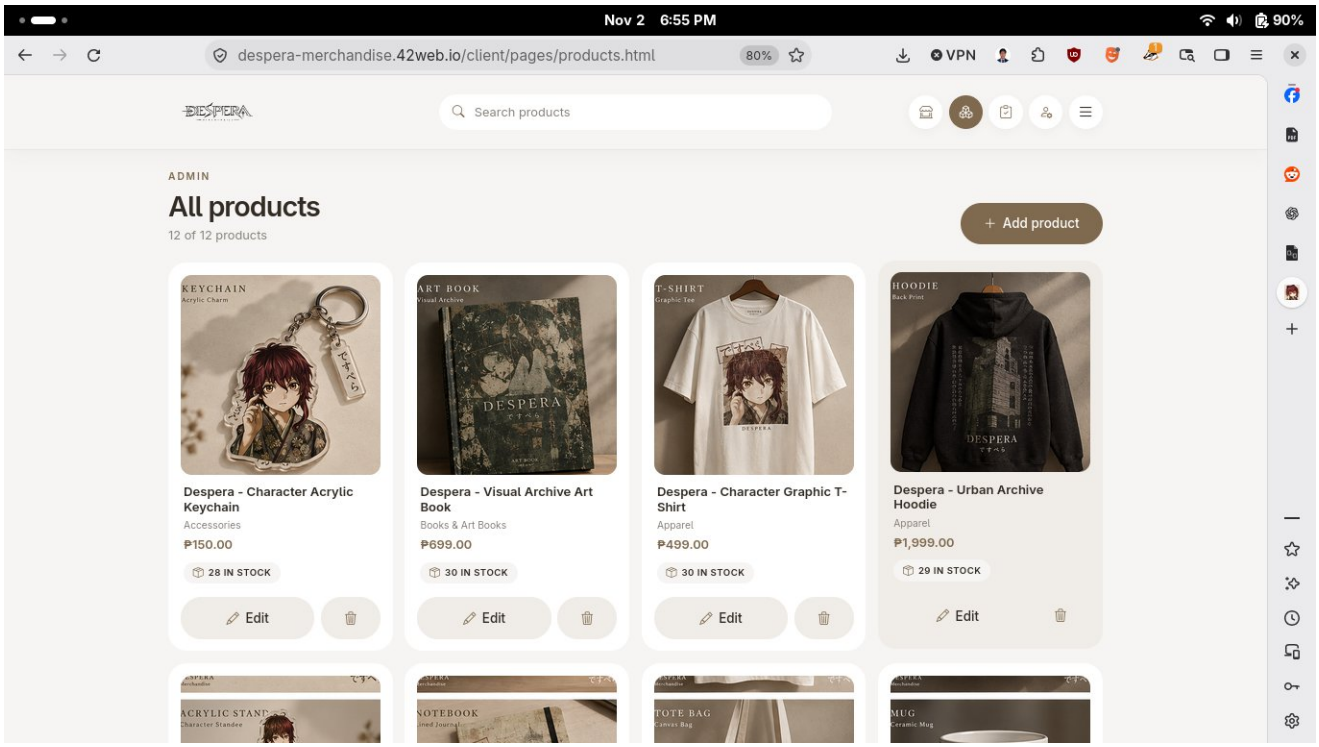


Figure 7. Product page.

VIEW/ADD/DELETE/UPDATE USERS

Navigate to Users page and view users, Click add user to add user, Click Delete Icon to delete user, Click edit button to update user.

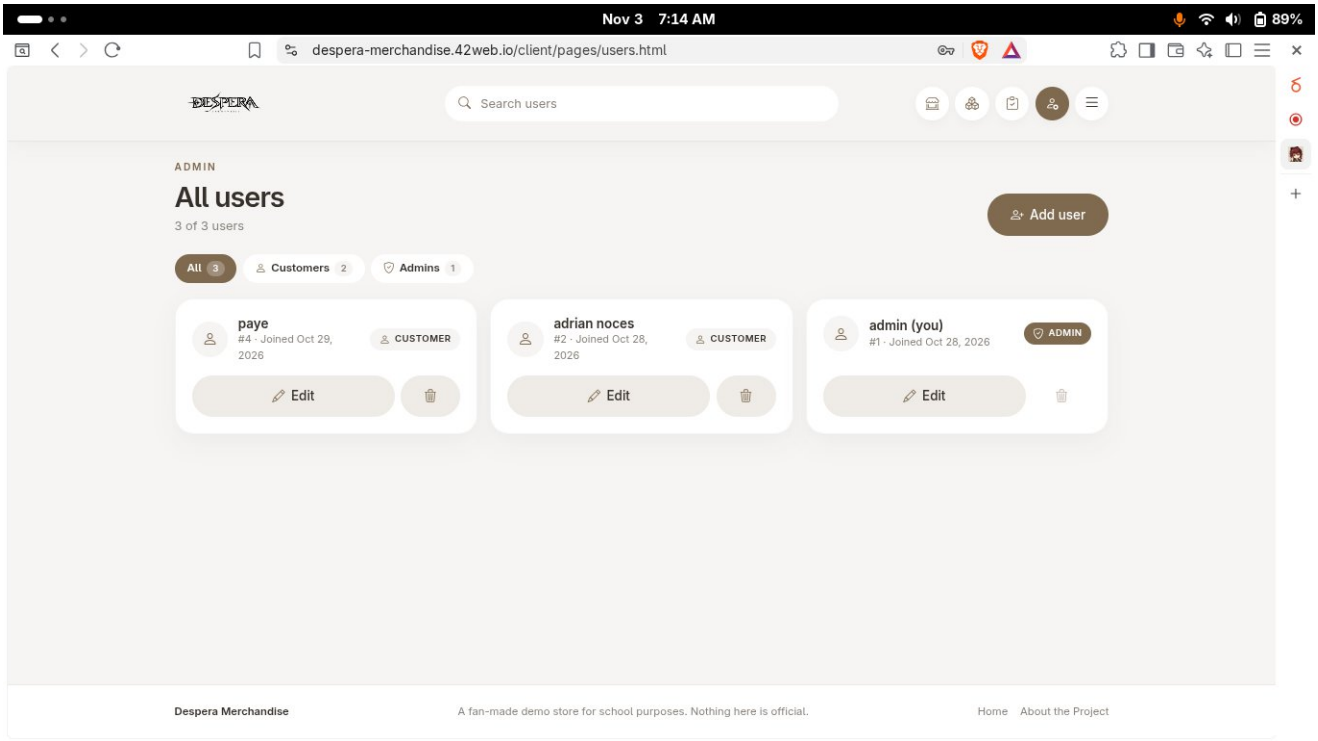


Figure 8. Users page.

SEARCH

Click search bar to start searching.

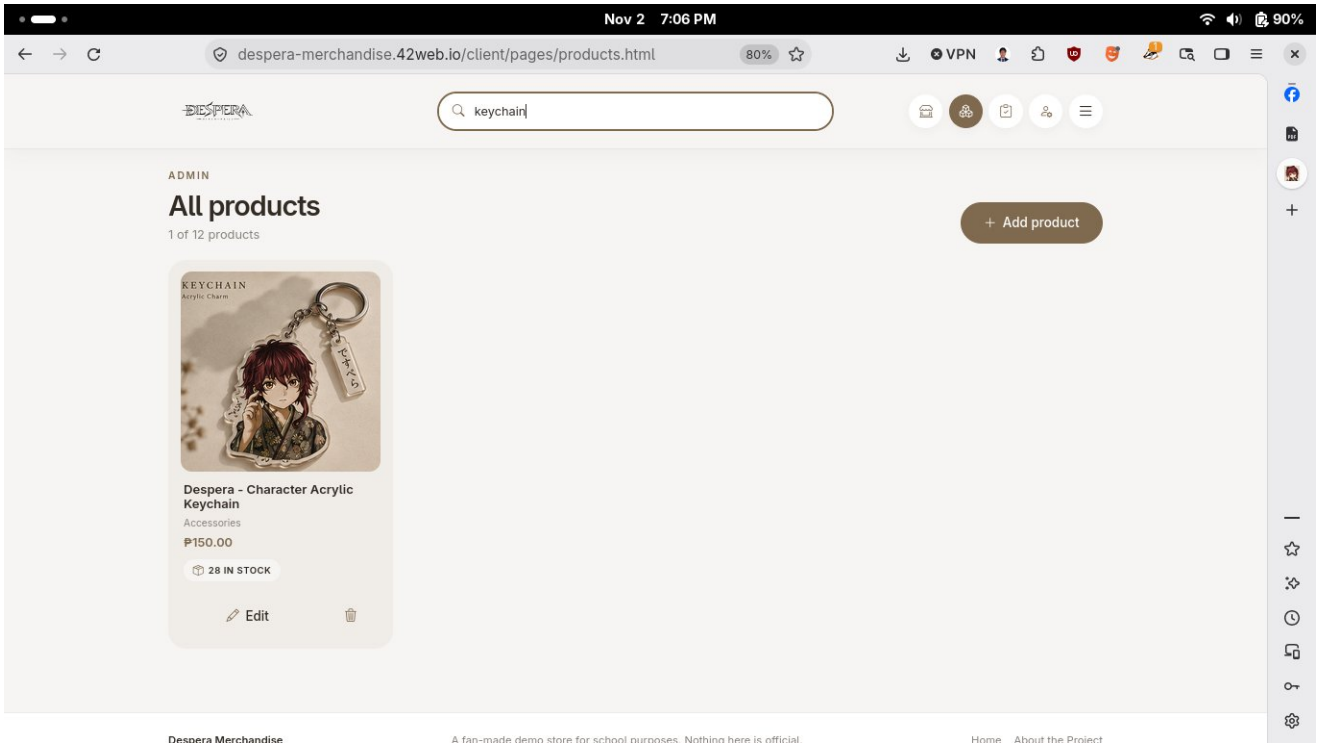


Figure 9. Search products.

VIEW/UPDATE Profile

Click search bar to start searching.

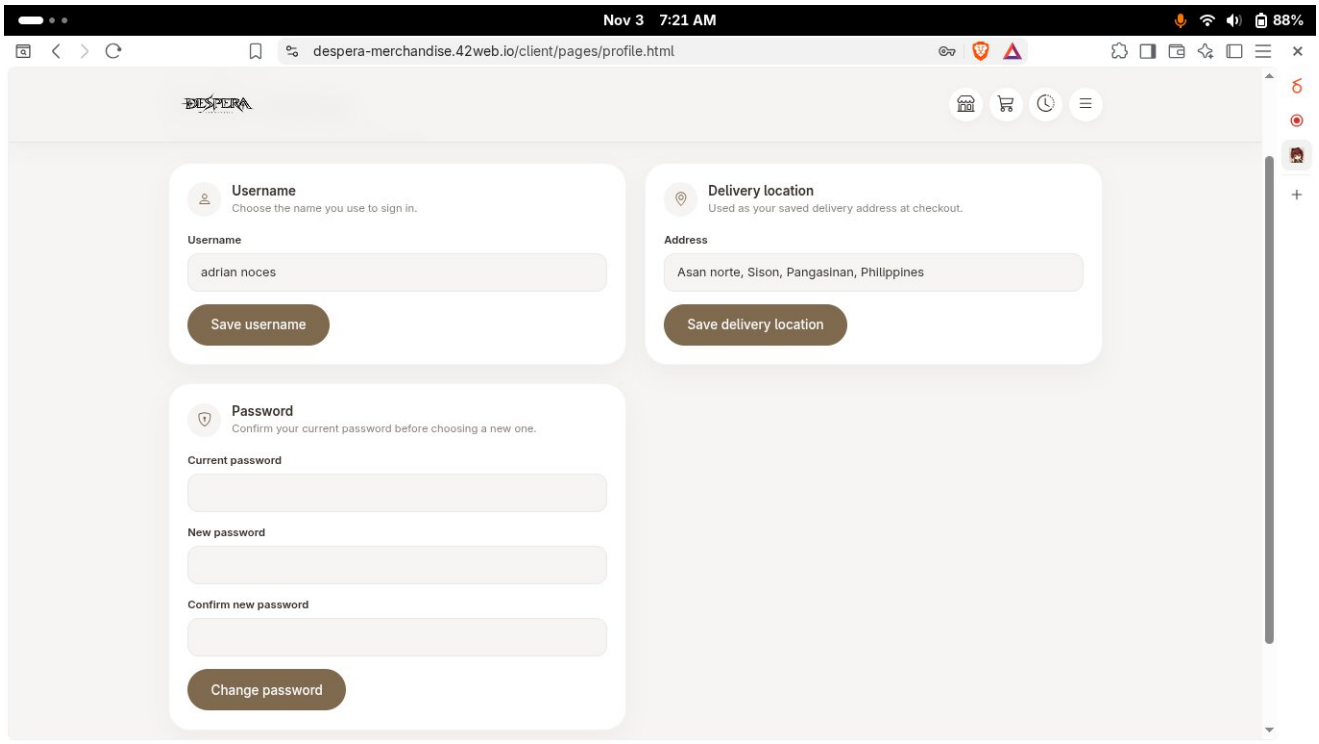


Figure 10. Profile page.

SECTION IV

TESTING

The repository has no automated functional test suite. The table below is a functional acceptance checklist derived from implemented routes and page behavior, updated with read-only checks of the live deployment. Only the public catalog reads and anonymous session check were exercised on 2 November 2026. Actual results for authentication with valid credentials, CRUD mutations, checkout, payment, and administrator actions remain Not run because no test account was supplied.

Test case	Expected result	Actual result	Status
Login: valid customer	Authenticated session is created and customer page opens.	Successful	Passed
Login: invalid password	Sign-in is rejected with a clear error; no session is created.	Error Displayed actual error	Passed
Add/create product	Authorized admin saves valid product and it appears in the catalog.	Successful	Passed
View/read products	Catalog returns products and details reflect stored values.	Records Viewed Successful	Passed
Search products	Matching products remain visible; nonmatches are filtered.	Record Search Successful	Passed
Edit/update product	Authorized admin saves changes; updated fields display.	Record Updated	Passed
Delete product	Admin confirms deletion and product is removed from catalog.	Record Deleted	Passed
Cart and checkout	Non-empty cart creates an order with correct item snapshots and total; cart clears.	Successful	Passed
Empty-cart checkout	Order creation is rejected and no duplicate/empty order is stored.	Successful	Passed
Demo payment / stock	Pending order with available stock is paid once; inventory is decremented; insufficient stock is rejected.	Successful	Passed
Order history / ownership	Customer sees own orders only; another user's order cannot be altered.	Successful	Passed
Admin authorization	Customer cannot access admin operations; admin can access permitted management routes.	Customer cant access admin pages	Passed
Customer-service cooldown	Conversation is retained and a second request inside the shared cooldown is delayed/rejected as designed.	Chat cool down store wide	Passed

SECTION V
CONCLUSION

What the Group Developed

The group developed a B2C merchandise storefront and administration system branded Despera Merchandise. It connects a responsive web client, PHP API, MySQL persistence, customer and administrator sessions, product/category CRUD, customer carts and profiles, checkout/order history, administrative summaries, reports, and an optional AI-assisted customer-service module.

What Was Achieved

The implementation models the main shopping lifecycle and the related administrative work using shared database records. It provides user-owned sessions and carts, catalog and order endpoints, role-checked admin routes, price/item snapshots, and transactional stock deduction when an order is marked paid. Deployment notes explain local container setup and static frontend hosting. The AI experience is optional and its external-service limitations are disclosed.

Problems Encountered and Solutions Implemented

A full e-commerce flow must keep stock, order state, and item totals consistent when requests overlap or a customer repeats an action. The implementation uses order-state checks, a transaction and row locks during payment, and conditional stock updates. Session identity and administrator permissions also need to remain server-controlled; the API validates the cookie-backed session and checks role/ownership on protected operations. External AI and hosted frontends add configuration and availability dependencies, so AI is optional, a database-backed fallback notice exists, and deployment/CORS guidance is documented.

Lessons Learned

- A B2C workflow crosses browser state, authorization, database relationships, and business rules; each layer must agree on ownership and status.
- Preserving order-time item details and handling inventory inside a transaction are important for reliable order records.
- Role-specific interfaces are not a substitute for server-side authorization.
- Integration services introduce privacy, availability, rate-limit, and configuration responsibilities that must be disclosed to users.
- Documentation should distinguish code-derived behavior from execution evidence; tests and screenshots must be captured against a running, configured environment.

Possible Future Improvements

- Integrate a verified payment provider with signed callbacks, refunds, reconciliation, and clear order/payment state separation.
- Add automated API and browser tests for authentication, CRUD, authorization, checkout, stock contention, and error cases; include a repeatable test database.
- Add stronger deployment controls such as HTTPS-only production configuration, rate limiting, secret management, backups, audit logging, and operational monitoring.
- Improve order fulfillment with shipment details, notifications, cancellation/refund policy, and a clearer lifecycle beyond payment.
- Provide accessible keyboard/focus behavior and evaluate all responsive layouts with real devices and assistive technology.
- Add product/category pagination and server-side search for larger catalogs; validate image storage limits and lifecycle.
- Document AI data handling in the UI, allow conversation deletion, and review retention/privacy settings before any production use.